# 电子科技大学

# 实验报告

**课程名称：高级计算机图形学**

**学　　院：计算机科学与工程学院**

**专　　业：计算机科学与技术**

**指导教师：蔡洪斌**

**学生姓名：张瀚文**

**学　　号：2024030901009**

**实验成绩：**

**日　　期：2026 年 11 月 3 日**

# 电 子 科 技 大 学

# 实 验 报 告

## 一、实验项目名称：实验二-纹理映射

## 二、实验室名称：主楼 A2-402　　实验时间：第七周周日

## 三、实验目的：

掌握 OpenGL 中纹理创建、纹理坐标设置和采样参数配置的方法，完成走廊场景的纹理映射。通过六种缩小过滤方式的对比，理解 Mipmap 的层级结构、层内插值和层间插值，分析远处纹理走样与细节保留之间的关系。

实现基于同一张 HDR 全景图的背景和镜面环境反射，理解观察方向、表面法线与反射方向的关系；将 Mipmap 设置和环境映射接入已有的三维模型查看器。

## 四、实验原理：

### 1 纹理映射与纹理过滤

纹理映射用二维纹理坐标将图像颜色赋给几何表面。走廊由左右墙壁、地板、天花板和尽头端面组成，每个顶点同时提交位置和纹理坐标。材质纹理采用 GL_REPEAT，使砖墙和地板沿走廊重复排列；尽头照片只完整显示一次。透视投影下，远处相同面积的表面会投影到更少的屏幕像素。

当一个屏幕像素覆盖多个纹素时，只从原图取最近邻或做双线性插值，仍难以代表整个覆盖区域的平均颜色，因此高频纹理会出现摩尔纹等走样。Mipmap 预先建立不同分辨率的纹理，使远处像素能从更低分辨率的图层取样。

### 2 Mipmap 的层级与存储量

任务 1 将 512×512 的原图逐级缩小为 256×256、128×128，直到 1×1，共 10 级。每一级采用 BOX 面积平均缩小，并通过 glTexImage2D 上传。二维完整链的纹素数约为原图的 4/3，即额外增加约 1/3 的纹素存储量。

$$512^2 + 256^2 + \ldots + 1^2 = 349525 \approx 512^2 \times 4/3$$

过滤名称中的第一个 NEAREST 或 LINEAR 表示层内采样方式；MIPMAP 后面的 NEAREST 或 LINEAR 表示层级选择或层间插值方式。放大过滤只使用 GL_NEAREST 或 GL_LINEAR。

## 四、实验原理（续）

| 缩小过滤方式 | 层内处理 | 层间处理 |
| --- | --- | --- |
| GL_NEAREST | 原始图最近邻 | 不使用 Mipmap |
| GL_LINEAR | 原始图双线性 | 不使用 Mipmap |
| GL_NEAREST_MIPMAP_NEAREST | 最近邻 | 选最近一级 |
| GL_LINEAR_MIPMAP_NEAREST | 双线性 | 选最近一级 |
| GL_NEAREST_MIPMAP_LINEAR | 最近邻 | 相邻两级线性插值 |
| GL_LINEAR_MIPMAP_LINEAR | 双线性 | 相邻两级线性插值 |

三线性过滤在相邻两级中分别做双线性采样，再按层级权重插值，能缓和层级切换。它会平滑远处的细节；走廊墙面斜视时，像素在纹理空间的覆盖区域被拉长，各向同性 Mipmap 仍可能留下走样。

### 3 环境映射的反射方向

环境映射把周围场景近似为无限远的方向颜色场。对于世界坐标中的表面位置 P、相机位置 C 和单位法线 N，程序先计算从相机指向表面的入射方向 I，再求反射方向 R。

$$I = \text{normalize}(P - C) \qquad R = I - 2\,\text{dot}(N, I)\,N$$

球体的法线随位置连续变化，反射方向也连续变化；立方体每个面使用独立法线，因此棱边处的反射出现折变。立方体同一个面上的观察向量也会随片元位置改变，面内颜色并非必然相同。

### 4 全景坐标与 HDR 显示

设归一化反射方向为 d，采用 Y-up 方向约定，将其转换为等距柱状全景图坐标。背景使用相机视线方向，物体使用反射方向，两者共用同一个方向到 UV 的函数。

$$u = \text{atan2}(dz, dx) / (2\pi) + 0.5 \qquad v = \text{asin}(dy) / \pi + 0.5$$

任务 2 直接读取 EXR 线性浮点数据，翻转图像行方向后上传为 RGB16F 纹理。背景和物体统一乘以 2^EV，再进行 ACES 近似色调映射和 sRGB 转换，避免同一环境在不同对象上出现不一致的亮度。经度方向使用重复采样，纬度方向使用边缘钳制。

## 五、实验内容：

编程任务 1：搭建由砖墙、木地板和灰泥天花板组成的透视走廊，生成材质纹理和高频棋盘纹理，建立完整 Mipmap 链。在尽头补充一张 3:2 的照片，封闭原本未绘制的空白区域。通过右键菜单、下拉框和数字键 1—6 选择过滤方式，使用键盘或鼠标移动、环顾场景。

编程任务 2：读取 meadow_2_4k.exr，将同一张全景纹理同时用于背景、球体和立方体的镜面反射。支持立方体显隐、相机远近调节、环绕观察、曝光调整和环境旋转。

进阶任务：在已有三维模型查看器中增加 MipmapPlugin 和 EnvironmentMappingPlugin，分别控制普通贴图的过滤参数，以及模型反射和全景背景。实验以提交 2e22f8a 为基础，补充了任务 1 的尽头照片端面。

## 六、实验器材（设备、元器件）：

| 项目 | 配置或用途 |
| --- | --- |
| 计算机与图形设备 | Intel Arc 130T GPU；OpenGL 4.6.0 |
| 软件环境 | Windows；Python 3.10.11；PyQt5 5.15.11 |
| 图形与数值库 | PyOpenGL 3.1.10；NumPy 2.2.6；Pillow 12.3.0 |
| 全景读取与查看器 | OpenEXR 3.5.2；VTK 9.5.0 |
| 实验资源 | 4 张生成纹理；1 张 3:2 照片；4096×2048 EXR 全景图 |

## 七、实验步骤：

### 1 准备环境并运行程序

在仓库已有虚拟环境中检查依赖。两个基础任务为独立桌面程序，无需先启动模型查看器。任务 1 请求 OpenGL 2.1 接口；任务 2 请求 OpenGL 3.3 兼容上下文，并使用 GLSL 1.30 着色器。

```
.venv\Scripts\python.exe -m pip install -r 实验 2\requirements.txt
.venv\Scripts\python.exe 实验 2\task1_mipmap.py
.venv\Scripts\python.exe 实验 2\task2_environment.py
```

启动后先确认走廊纹理与全景背景正常显示，再进行过滤切换、相机移动和显隐检查。截图时固定相机位置、窗口大小及其他参数，避免将视角变化误认为过滤方式带来的差异。

## 七、实验步骤（续）

### 2 走廊建模与纹理生成

在 task1_mipmap.py 中，CorridorView 负责 OpenGL 初始化、场景绘制和相机交互，MipmapWindow 负责控件和状态同步。make_textures 使用固定随机种子 2026 生成砖墙、木地板、灰泥和棋盘四张 512×512 纹理，保证不同过滤方式使用相同的输入图像。

走廊范围为 x∈[−3,3]、y∈[−2,2]、z∈[−100,6]。左右墙使用相同砖墙纹理，地板和天花板分别绑定对应材质；z=−100 处增加照片端面，纹理坐标覆盖[0,1]。初始化时为每张纹理上传完整层级，避免使用 Mipmap 过滤时因缺层而形成不完整纹理。

### 关键源程序 1 完整 Mipmap 链的上传

```
# 每次循环上传一级，直到最小尺寸 1×1。
image, level = Image.fromarray(pixels), 0
while True:
    gl.glTexImage2D(gl.GL_TEXTURE_2D, level, gl.GL_RGB,
                    image.width, image.height, 0, gl.GL_RGB,
                    gl.GL_UNSIGNED_BYTE, np.asarray(image))
    if image.size == (1, 1):
        break
    image = image.resize((max(1, image.width // 2),
                          max(1, image.height // 2)), Image.Resampling.BOX)
    level += 1
```

### 3 过滤参数与相机控制

每次绘制表面前，依据 filter_index 设置 GL_TEXTURE_MIN_FILTER。对应放大方式只设为最近邻或双线性，避免给 GL_TEXTURE_MAG_FILTER 传入 Mipmap 枚举。菜单、下拉框和数字键共用 set_filter，减少控件状态与渲染状态不一致的问题。

```
gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER,
                   FILTERS[self.filter_index][1])
mag = gl.GL_NEAREST if self.filter_index in (0, 2, 4) else gl.GL_LINEAR
gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, mag)
```

W/S 和上下键前后移动，A/D 和左右键横向移动，Q/E 升降，左键拖动调整朝向，滚轮前后移动。相机位置限制在走廊内部，R 键恢复初始位置。比较时先重置相机，再逐项切换过滤模式。

## 七、实验步骤（续）

### 4 HDR 全景图加载与对象绘制

在 task2_environment.py 中，load_panorama 负责读取和整理 EXR 数据；EnvironmentView 管理全景纹理、着色器、球体和立方体；EnvironmentWindow 提供曝光、旋转、显隐和文件选择控件。读取时处理 RGB、RGBA 或分离通道，清理非有限值，并翻转一次图像上下方向。

全景图使用 GL_RGB16F 存储并生成 Mipmap。球体用 128 个经向分段和 80 个纬向分段构造，顶点法线等于单位球方向；立方体为 6 个独立面，顶点和法线经过相同旋转，再平移到球体右侧。

### 关键源程序 2 逐片元计算环境反射

```
// worldPosition 与 worldNormal 均位于世界坐标系。
vec3 incident = normalize(worldPosition - cameraPosition);
vec3 reflected = reflect(incident, normalize(worldNormal));
gl_FragColor = vec4(environment(reflected), 1.0);
```

背景先关闭深度测试和深度写入，以全屏四边形生成观察射线；物体绘制前恢复深度状态。背景和物体程序都调用 use_environment，绑定 self.texture 到纹理单元 0，因此使用同一个 GPU 纹理对象和相同的曝光、旋转参数。

### 关键源程序 3 全景接缝的导数处理

```
vec2 dx = dFdx(uv), dy = dFdy(uv);
// 经度跨越 0/1 时，将差分折回较短的环绕距离。
dx.x -= floor(dx.x+0.5);
dy.x -= floor(dy.x+0.5);
float footprint = max(length(dx*vec2(textureSize)),
                      length(dy*vec2(textureSize)));
float lod = max(0.0, log2(max(footprint, 0.0001)));
vec3 color = texture2DLod(panorama, uv, lod).rgb * exp2(exposure);
```

经度坐标由 atan 计算，在全景接缝处从 1 跳到 0。若直接用这个跳变计算纹理覆盖范围，会误选过低分辨率的层级，形成模糊竖线。代码先修正经度差分，再计算 LOD；它处理的是接缝引起的导数异常，并不能消除全景图极点附近的投影畸变。

## 七、实验步骤（续）

### 5 进阶插件的接入

MipmapPlugin 在模型设置区域增加过滤方式下拉框，并注册渲染窗口的 StartEvent。渲染前检查当前模型是否有普通贴图，启用 Mipmap、加载纹理，再直接设置 vtkTextureObject 的缩小和放大参数。销毁插件时移除观察者，避免残留回调。

**关键源程序 4 更新 VTK 底层纹理参数**

```
texture.InterpolateOn()
texture.MipmapOn()
texture.Load(self.ctx.vtk_renderer)
obj = texture.GetTextureObject()
if obj is not None:
    _, min_filter, mag_filter, _ = FILTERS[self.filter_index]
    obj.SetMinificationFilter(min_filter)
    obj.SetMagnificationFilter(mag_filter)
    obj.SendParameters()
texture.PostRender(self.ctx.vtk_renderer)
```

EnvironmentMappingPlugin 用独立的背景渲染层绘制全景，并以片元着色器替换实现模型镜面采样。模型反射方向从视图坐标转换到世界坐标后，使用与背景相同的全景 UV 公式。开启反射前保存原材质、纹理、映射器和着色器，关闭时恢复这些状态。

插件与独立任务 2 的颜色处理有所不同：插件在 CPU 端先对 EXR 做 ACES 近似映射，再生成共享浮点纹理，着色器使用近似 gamma 转换；独立程序保留 HDR 数据到 GPU，采样后再进行曝光、色调映射和分段 sRGB 转换。两条路径分别保证背景与反射共享各自的颜色处理。

### 6 调试检查与问题处理

纹理完整性：检查 10 级纹理是否全部上传，以及缩小、放大参数是否分别设置。只改变菜单文字而未更新底层参数，不能证明过滤方式已经切换，因此进阶任务直接读取纹理对象的过滤值进行核对。

反射方向与图像朝向：统一 Y-up 约定，确认图像只做一次上下翻转；观察向量和法线在相同坐标系中运算。球体和立方体均使用同一 environment 函数，避免各自采用不同的经纬度方向。

资源与状态：加载新全景图时，先完成解码再替换旧纹理；背景绘制后恢复深度状态。插件关闭反射时恢复普通材质，避免环境映射设置影响后续模型显示。

## 八、实验数据及结果分析：

### 1 走廊材质映射结果

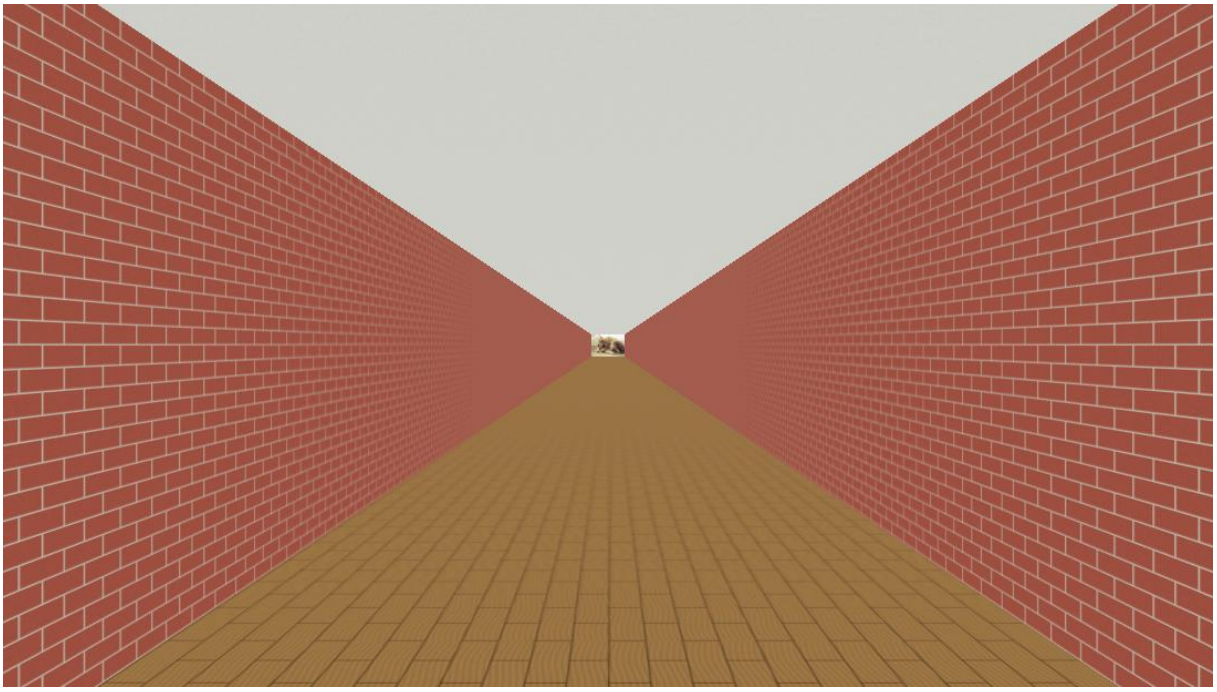


图 1 砖墙 木地板和灰泥天花板的纹理映射结果

图 1 采用 GL_LINEAR_MIPMAP_LINEAR。近处砖缝和木板纹理能够辨认，纹理沿走廊重复排列；随着距离增加，砖缝逐渐合并，远处表面更接近平均颜色。纹理并未随距离丢失绑定，而是因透视缩小和低分辨率层级采样而减少了可见细节。

| 记录项目 | 数值 |
| --- | --- |
| 初始相机位置 | (0, 0, 4) |
| 按一次 W 后的相机位置 | (0, 0, 3.4) |
| 按 R 重置后的相机位置 | (0, 0, 4) |
| 透视视场角与走廊长度 | 60°；106 个场景单位 |
| 纹理数量及层级 | 4 张 512×512 生成纹理，各 10 级；另加 1 张照片 |

相机状态记录表明，W 键使观察点沿走廊前移 0.6 个单位，R 键能恢复初始位置。纹理坐标固定在场景表面，相机移动改变投影比例，从而让同一张纹理在不同距离上进入不同的缩小程度。

为单独观察采样差异，对比截图将四个长表面切换为 8×8 纹素一格的高频棋盘纹理，并保持相机位置、视角和渲染尺寸一致。尽头照片始终保留，便于确认端面已封闭。

## 八、实验数据及结果分析（续）

### 2 走廊尽头的照片纹理

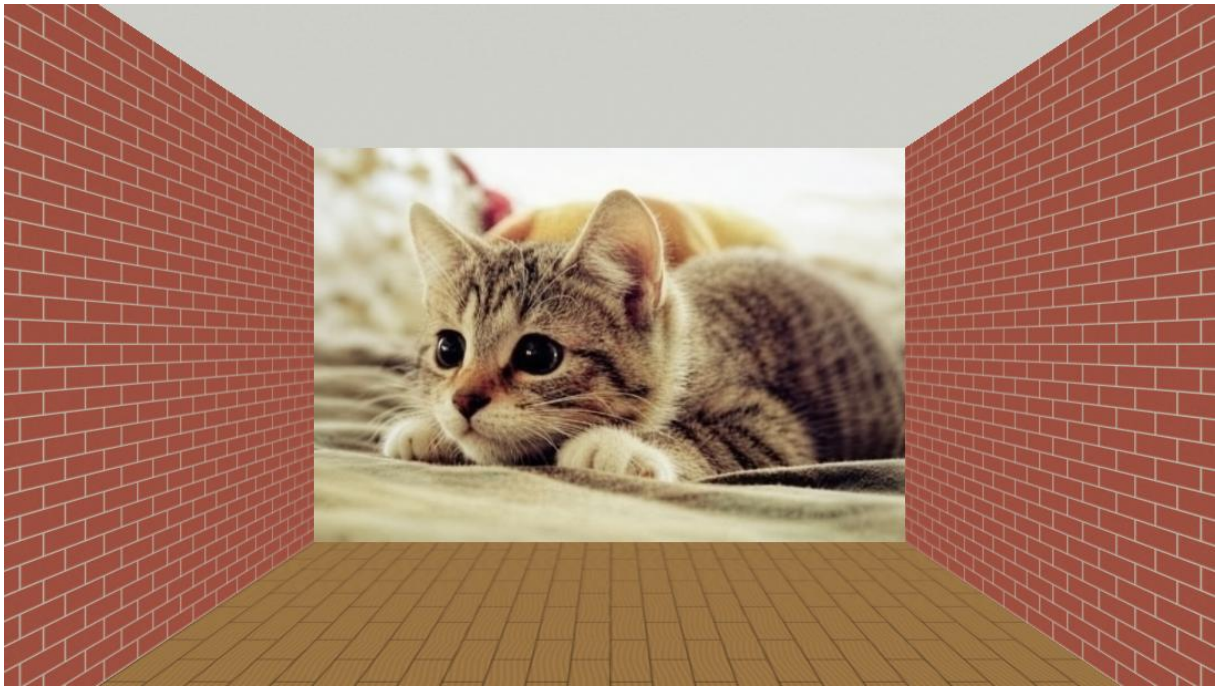


图 2 接近走廊尽头时的照片端面与周围材质

尽头原本没有几何面，远处的小块黑色区域实际显示的是清屏背景。在 z=−100 处增加宽 6、高 4 的四边形后，使用猫的照片覆盖整个端面。图片扩展为横向 3:2，与端面长宽比一致，避免将原正方形照片直接拉宽。

照片纹理从脚本目录加载，上传前翻转一次图像行方向。端面的 UV 仅覆盖[0,1]，不乘墙面使用的重复系数，并将 S、T 方向设为 GL_CLAMP_TO_EDGE。切换棋盘纹理时，只替换墙壁、地板和天花板，保留尽头照片。

```
# 尽头四边形与 UV 对应：左下、右下、右上、左上。
vertices = ((-3, -2, -100), (3, -2, -100),
            (3, 2, -100), (-3, 2, -100))
uv = ((0, 0), (1, 0), (1, 1), (0, 1))
# 照片使用独立纹理，仍参与当前缩小过滤与 Mipmap 采样。
selected = 3 if self.checker and texture_index != 4 else texture_index
repeat = 1.0 if texture_index == 4 else self.repeat
```

## 八、实验数据及结果分析（续）

### 3 六种过滤方式的对比

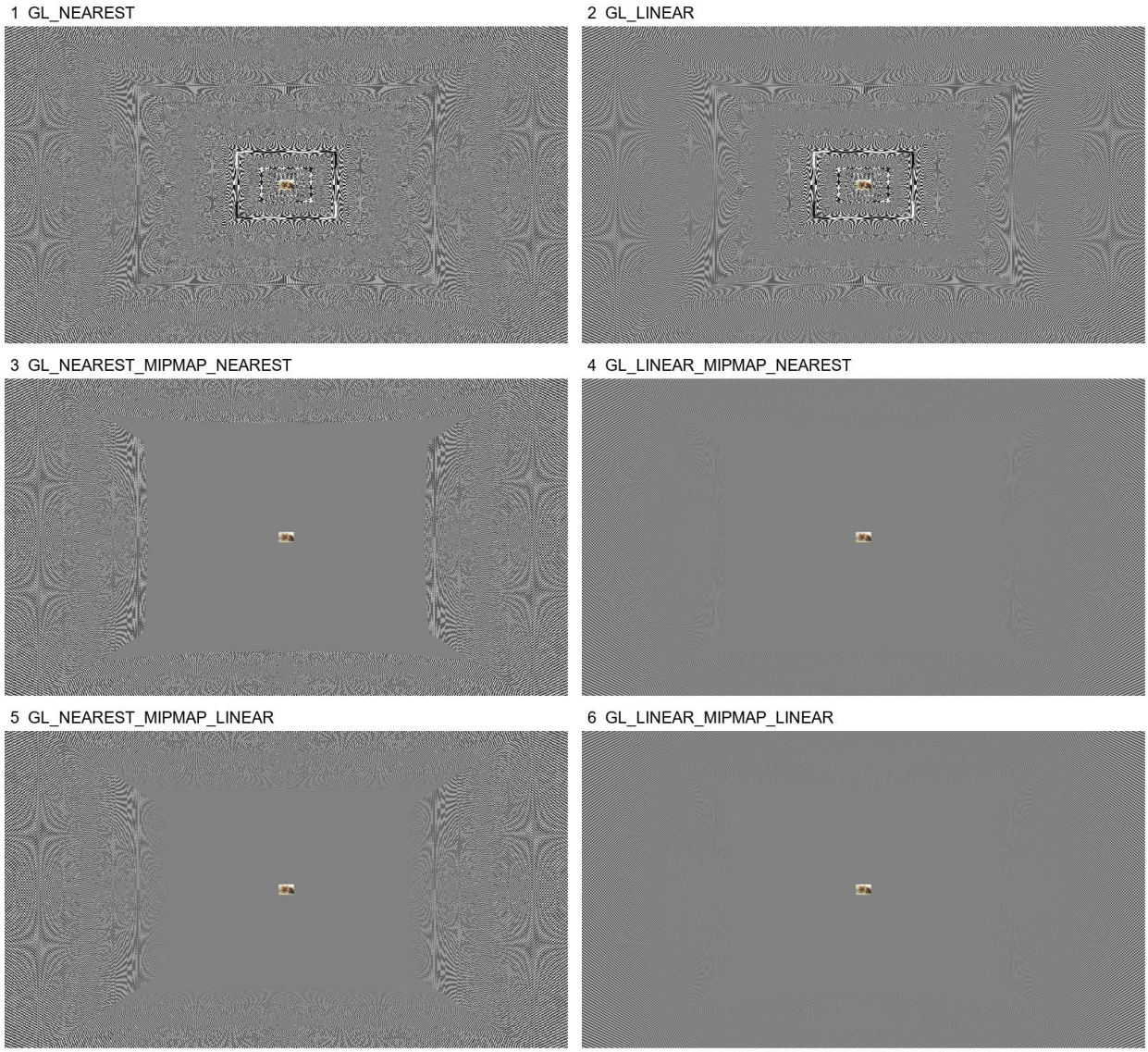



图 3 同一视角下六种过滤方式的高频棋盘纹理对比

图 3 前两种方式始终从原图采样，远处形成密集的明暗条纹和摩尔纹。双线性插值平滑了局部边缘，但没有充分平均一个远处像素所覆盖的大量纹素。启用 Mipmap 后，远处大面积区域趋于灰色，说明更低分辨率图层提供了平均后的颜色。

选择最近一级时，细节区域与灰色区域之间的层级边界较明显；相邻级线性插值使过渡更平缓。三线性过滤的过渡最柔和，但近侧墙面仍有细密条纹，说明斜视时仍存在走样。静态截图能显示空间走样和层级差异；移动时的闪烁还需要结合连续画面观察。

## 八、实验数据及结果分析（续）

### 4 环境映射结果与几何形状的影响



图 4 同一全景图用于背景 球体和立方体的镜面环境映射

球体上能看到天空、树木和草地的连续弯曲倒影，立方体各面的倒影在棱边处折变。立方体顶面主要反射天空，正面呈现草地和树木，差异来自法线方向。背景与物体共享纹理，并不意味着画面中同一屏幕位置必须显示同一颜色，因为它们采样的方向不同。



图 5 隐藏立方体后的环境映射结果

隐藏立方体后，球体和全景背景仍正常显示。该方法从无限远环境取样，不计算物体之间的相互反射；因此不能用球体上是否出现立方体倒影来判断显隐功能是否正确。

## 八、实验数据及结果分析（续）

### 5 交互功能与进阶插件检查

| 检查项 | 记录或结果 |
| --- | --- |
| 任务 2 全景尺寸 | 4096×2048 |
| 任务 2 默认相机距离 | 7.8 个场景单位 |
| 按一次“+”后的距离 | 6.864，即 7.8×0.88 |
| R 键重置距离 | 恢复为 7.8 |
| Mipmap 插件六种选项 | 读取的缩小及放大参数均与枚举配置一致 |
| 环境插件纹理共享 | 背景与模型材质绑定同一 vtkTexture 对象 |
| 关闭模型反射 | 原普通贴图恢复 |

相机拉近通过改变观察距离实现，物体投影随之变大；背景射线仍由相机朝向和视场角确定。环境旋转同时作用于背景和物体的取样方向。曝光也共用于背景和物体，避免只改变某一部分的显示亮度。

独立程序的环境纹理保留 HDR 动态范围，插件则在上传前完成色调映射。两者均属于镜面环境映射，没有加入粗糙度驱动的反射预过滤、局部遮挡或物体间反射；本实验的结果不能等同于完整的物理渲染。

## 九、总结、改进建议及心得体会：

本次实验把纹理映射中容易混在一起的两个问题分开了：纹理坐标决定表面到图像的对应关系，过滤方式决定像素从图像中怎样取色。走廊中的高频棋盘表明，双线性过滤即使让局部边缘变平滑，也不能充分解决远处缩小时的走样。Mipmap 通过预先平均纹理降低高频变化，代价是远处细节变软；三线性过滤进一步平滑了层级过渡。

环境映射中，法线和观察方向共同决定反射方向。球体法线连续，立方体法线在棱边处分段，因此即使使用同一张全景图，两种几何体的反射也有明显差别。背景与反射是否协调，需要同时检查方向约定、纹理来源和颜色处理，不能只看它们是否读取同一个文件。

后续可加入各向异性过滤，改善走廊斜视角下仍存在的条纹；也可为环境反射增加粗糙度和预过滤层级，使材质从清晰镜面过渡到模糊反射。插件部分可继续统一 HDR 颜色处理流程，并增加相机预设，便于在固定条件下重复比较。

### 源程序清单

完整源程序及原有注释随附件“实验二源程序清单（含尽头贴图）.zip”提供，包含两个独立程序、两个进阶插件和尽头图片资源。任务 1 在提交 2e22f8a 的基础上补充照片端面，其余程序保留该提交的实现。